

As can be seen in the Student model above, we have three fields: a character field, an integer field and a JSONField. Let us enter three types of data in our JSONField: an empty value, a dictionary and a list of dictionaries. With the help of the code snippets given in Figures 1, 2 and 3, let's see how the two points mentioned in the beginning come into play.

Unlike our 'StudentSerializer', which strictly checks the presence of and types of values entered for the 'student_name' and 'roll_no' fields within our Student model (while merely checking if the JSONField 'detailed_data' exists regardless of the type of data entered for it, as observed in Figures 1, 2 and 3), we do not currently possess a defined format that our JSONField must follow. This is where the keyword 'schema' kicks in. Schema is the outline or the structure that the data within JSONField needs to abide by. If the data/values we enter into 'detailed_data' in Insomnia abide by the schema, then we can say that the validation of 'detailed_data' is

successful. However, if the value entered for a field within the 'detailed_data' is of a type different from the one defined in the schema, or if any field(s) within 'detailed_data' is/are missing, it implies that we have encountered validation error(s). Let's try to understand this by deep-diving into the 'detailed_data' field within our Student model.

Let's follow the layout given below for the fields within our 'detailed_data' JSONField:

1. Date of joining — Date format
2. Mother's name — Character type
3. Father's name — Character type
4. Teacher information — A dictionary comprising: Name, Date of birth, Department, Date of joining
5. Semester 1 subjects — A list of dictionaries, with each dictionary comprising subject name, score
6. Semester 2 subjects — Another list of dictionaries with the same structure and subfields as mentioned in point 3.

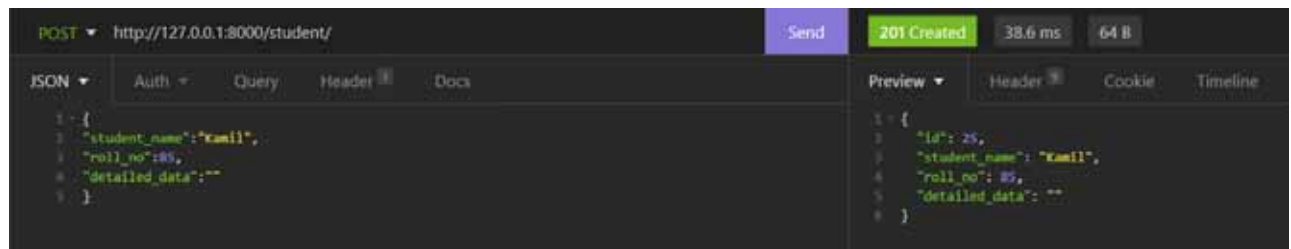


Figure 1: When entering an empty value into our JSONField 'detailed_data'

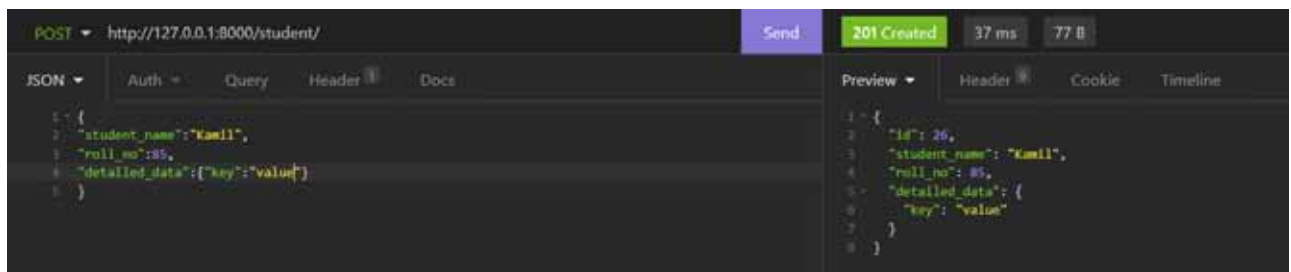


Figure 2: When entering a key-value pair into our JSONField

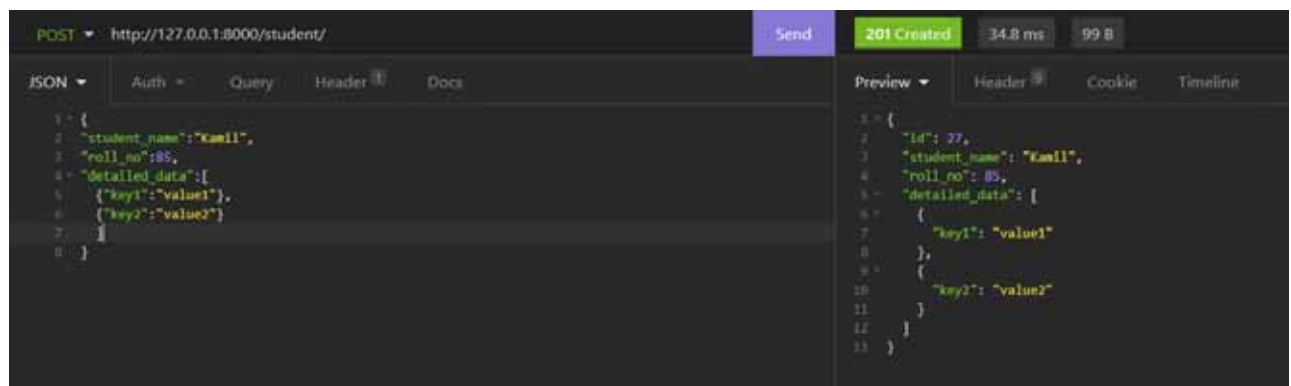


Figure 3: When entering a list of dictionaries